

USER MANUAL · VERSION 0.1.0

ShiaQAService

*Retrieval-augmented question answering over WikiShia,
running entirely on a local Ollama*

Command line · HTTP · Python

Complete API signature reference

Apache-2.0 (code only) · Python $\geq$ 3.11

Generated 24 August 2026

§ Contents

1 About this manual

2 Requirements and installation

- 2.1 Python environment
 - 2.2 Ollama and models
 - 2.3 Verifying the install
-

3 First run: corpus, index, graph

- 3.1 Fetching the corpus
 - 3.2 Building the index
 - 3.3 Building the knowledge graph
 - 3.4 The one destructive command
-

4 Everyday use

- 4.1 Asking questions
 - 4.2 Inspecting retrieval
 - 4.3 Exploring the graph
 - 4.4 Running the service
-

5 Command-line API

- 5.1 Corpus and index commands
 - 5.2 Query commands
 - 5.3 Knowledge-graph commands
-

6 HTTP API

- 6.1 The request model
 - 6.2 Endpoints
 - 6.3 Streaming event shapes
 - 6.4 Status codes and errors
-

7 Python API

- 7.1 shiaqa.config and shiaqa.log
 - 7.2 shiaqa.rag — answering
 - 7.3 shiaqa.rag.retrieve
 - 7.4 shiaqa.rag.decompose
 - 7.5 shiaqa.store.sqlite_store
 - 7.6 shiaqa.kg.store
 - 7.7 shiaqa.kg.paths
 - 7.8 shiaqa.kg — build, ontology, render, viz
 - 7.9 shiaqa.ingest and shiaqa.wiki
 - 7.10 shiaqa.embed and shiaqa.rag.prompt
-

8 Configuration reference

9 Evaluating retrieval changes

10 Troubleshooting

11 Licensing and attribution

1 About this manual

What the system is, who this manual is for, and how to read the signature blocks.

ShiaQAService answers questions about Shia Islam, its history, texts and figures from the [WikiShia](#) encyclopaedia. Everything runs on the machine in front of you: there are no API keys, and no question or document leaves the laptop. Answers are grounded in retrieved encyclopaedia passages and carry inline citations [1] , [2] back to the source articles.

MediaWiki API → raw JSONL cache → cleaned text → section-aware chunks
→ SQLite (rows + FTS5 + sqlite-vec) → hybrid retrieval → Ollama → cited answer

A second, independent pass over the same raw cache builds a structural **knowledge graph**: infobox fields are already typed relations and category pages are already a taxonomy, so the graph costs no model calls and about fifteen seconds. It feeds a fourth retrieval channel and can walk two typed hops to compose relations that appear in no single article.

The system exposes three APIs over the same engine, and this manual documents all three:

SURFACE	ENTRY POINT	USE IT WHEN
Command line	<code>shiaqa <command></code>	Building and maintaining the corpus; interactive questions; inspecting retrieval and the graph.
HTTP	<code>shiaqa serve</code>	Another process or a front end needs answers, streamed or whole.
Python	<code>import shiaqa...</code>	Embedding the retriever in your own code, or changing how retrieval works.

1.1 Conventions

Signatures are given exactly as they appear in the source, including type hints and defaults. A block looks like this:

```
async def answer(question: str, profile: str | None = None,
                  top_k: int | None = None,
                  decompose: bool | None = None) -> Answer
```

A short statement of what the call does, followed by anything about its arguments or return value that the signature does not already say.

Shell examples assume the project virtual environment. Either activate it, or prefix each command with `.venv/bin/` as the examples do. A leading `#` is a comment, never part of the command.

Names beginning with a single underscore are internal. They are documented here only where a reader modifying retrieval has to know they exist — they carry no stability promise.

1.2 Why the design looks the way it does

The target machine is an Intel Mac, where Ollama has no Metal path and therefore no GPU path at all. Everything is CPU-bound, and that single constraint explains most of what follows: SQLite instead of a vector server, a resumable embedding phase, a tight context budget, sources streamed before the answer, and a graph traversal that costs milliseconds rather than model calls. Where a knob exists because of that constraint, this manual says so.

2 Requirements and installation

Python 3.11 or newer, a running Ollama, two pulled models, and about 250 MB of disk.

2.1 Python environment

```
# from the project root
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -e "[dev]"
```

The package installs one console script, `shiaqa`, and the importable package `shiaqa` from `src/`. Runtime dependencies are `httpx`, `mwparserfromhell`, `sqlite-vec`, `numpy`, `FastAPI`, `uvicorn`, `Typer`, `Rich`, `Pydantic` and `pydantic-settings`; the `dev` extra adds `pytest` and `pytest-asyncio`.

2.2 Ollama and models

Install Ollama and pull the embedding model plus at least one generator:

```
ollama pull nomic-embed-text          # 768-d embeddings, required
ollama pull llama3.2:3b                # the "fast" profile
ollama pull qwen2.5:7b-instruct-q4_K_M # the "quality" profile
```

The service talks to Ollama over HTTP at `http://127.0.0.1:11434`. Any pulled tag can be used as a generator by passing it where a profile is expected; only the embedding model is fixed by the stored vectors.

DO NOT CHANGE THE EMBEDDING MODEL CASUALLY

Every stored vector was produced by `nomic-embed-text` with the prefix `search_document:`, and queries are embedded with `search_query:`. Changing `embed_model` or either prefix invalidates the whole vector index silently — retrieval degrades and nothing raises. Re-embedding is a multi-hour CPU job.

2.3 Verifying the install

```
.venv/bin/python -m pytest tests/ -q    # 171 tests, fully offline
.venv/bin/shiaqa status                  # index counts + Ollama health
```

The test suite needs neither Ollama nor the network nor a built index. If it passes but `status` reports that Ollama cannot be reached, the install is fine and the server is not running.

3 First run: corpus, index, graph

Three build steps, in order. The middle one takes hours; the other two take minutes.

3.1 Fetching the corpus

```
.venv/bin/shiaqa fetch
```

Downloads namespaces 0 (articles), 3000 (Text: primary sources) and 14 (categories) through the MediaWiki API into `data/raw/*.jsonl`, plus a redirect map into `data/raw/redirects.json`. It takes a few minutes and writes about 50 MB. Everything is cached: a second run is a no-op unless you pass `--force`.

The fetcher is deliberately a polite API client. It identifies itself with a contact address, serialises its requests, sleeps `request_delay` seconds between calls and honours `maxlag`. Do not parallelise it.

3.2 Building the index

Indexing has two phases because their costs differ by three orders of magnitude. Phase 1 — clean, chunk, write rows and the FTS5 BM25 index — takes about twenty seconds. Phase 2 embeds every chunk at roughly four chunks per second on CPU, so about 40,000 chunks take some hours.

```
.venv/bin/shiaqa index          # both phases (wipes first — see 3.4)
.venv/bin/shiaqa index --embed-only # resume phase 2 only, always safe
```

Phase 2 is resumable by construction: it processes only the rows returned by

`Store.chunks_missing_vectors()`, so it can be interrupted with Ctrl-C and restarted as often as you like. Progress is reported with a rate and an ETA computed over a trailing window, so a laptop that slept does not report a misleading average.

3.3 Building the knowledge graph

```
.venv/bin/shiaqa kg build      # ~15 s
```

Reads `data/raw/` only and writes `data/kg/*.jsonl` plus `data/kg.db`. It never opens `shiaqa.db`, so it cannot disturb the index or the embeddings — it is the one build command in this project you can re-run without thinking. The JSONL files are canonical and greppable; `kg.db` is a derived index rebuilt from them in seconds.

With the graph present and the default settings, retrieval gains a fourth channel and the prompt gains a `[KG-PATH]` block of composed relations. Without it, the system works exactly as it did before the graph existed.

3.4 The one destructive command

SHIAQA INDEX WITH NO FLAGS WIPES THE DATABASE

build_text_index() calls Store.reset() , which drops pages , chunks , chunks_fts , vec_chunks and meta – **including every embedding**. On this hardware that is a multi-hour loss.

INTENT	COMMAND
Resume or top up embeddings	shiaqa index --embed-only
Rebuild text + BM25, knowing it destroys vectors	shiaqa index --text-only
Drop navigation pages and orphan vectors, keeping embeddings	shiaqa prune
Delete specific pages from a live index	Store.delete_page(page_id)

Check shiaqa status before and after any index operation. The knowledge graph is immune to all of this: it lives in its own file precisely because Store.reset() drops everything shiaqa.db owns.

4 Everyday use

Four things you will actually do: ask, inspect retrieval, explore the graph, serve.

4.1 Asking questions

```
.venv/bin/shiaqa ask "Who compiled al-Kafi?"
.venv/bin/shiaqa ask "Who compiled al-Kafi?" --profile quality
.venv/bin/shiaqa ask "Compare the fourth and fifth Imams" --decompose
.venv/bin/shiaqa ask "Who compiled al-Kafi?" --json
```

The answer streams token by token. Sub-questions and reasoning chains, when there are any, print first; then the numbered sources; then the answer; then the model, token count and throughput; then the source URLs. `--json` switches to a single non-streaming JSON object instead, the same shape the HTTP `/ask` endpoint returns.

Expect roughly 14 s per answer on the `fast` profile and 27 s on `quality`. `--decompose` adds one model call before retrieval, so it costs about another six to eight seconds; it is off by default and worth it only for questions that genuinely span several articles.

4.2 Inspecting retrieval

```
.venv/bin/shiaqa search "Who compiled al-Kafi?" --full
```

Retrieval only, no generation, and therefore instant — about 109 ms median. This is the command to reach for when tuning: it prints the reasoning chains with the predicate trail behind each one, the graph facts, then every hit with its fused score and its rank in the vector and BM25 channels, and finally the context token count.

4.3 Exploring the graph

```
.venv/bin/shiaqa kg stats
.venv/bin/shiaqa kg show "Al-Shaykh al-Mufid"
.venv/bin/shiaqa kg chain "Al-Sharif al-Radi" --deep-only
.venv/bin/shiaqa kg path "Al-Shaykh al-Saduq" "Al-Shaykh al-Tusi"
.venv/bin/shiaqa kg compare "Al-Shaykh al-Mufid" "Al-Shaykh al-Tusi"
.venv/bin/shiaqa kg viz
```

All of these are pure SQLite: no model calls, ~10 ms at retrieval scale. `kg viz` writes two self-contained HTML pages to `data/kg/viz/` that open straight from disk with no server.

4.4 Running the service

```
.venv/bin/shiaqa serve --port 8000
```

Starts FastAPI under uvicorn. Interactive documentation is at `/docs` , and the endpoints are `/health` , `/search` , `/ask` and `/ask/stream` . The answering engine is constructed once at startup and closed at shutdown, so the first request does not pay for opening the database.

5 Command-line API

Every command, every flag, every default. Built with Typer; `--help` works on the root, on `kg`, and on each command.

```
shiaqa [COMMAND] [ARGS] [OPTIONS]
shiaqa kg [SUBCOMMAND] [ARGS] [OPTIONS]
```

Commands exit 0 on success. `search`, and every `kg` subcommand, exit 1 when they have nothing to show — no matches, no graph built, no entity by that name, no typed route — which makes them usable in shell conditionals.

5.1 Corpus and index commands

```
shiaqa fetch [--force] [--ns INTEGER]...
```

Download WikiShia into the local raw cache. Roughly 50 MB and a few minutes.

`--force` refetches even when the cache exists, and also refreshes `redirects.json`. `--ns` may be repeated to override the configured namespace list, e.g. `--ns 0 --ns 14`. Prints one line per namespace with the page count.

```
shiaqa index [--text-only] [--embed-only] [--limit INTEGER]
```

Build the search index. **With no flags this wipes the database first** (see §3.4) and then runs both phases.

`--text-only` runs phase 1 alone: clean, chunk, rows and BM25 — still destructive to existing vectors. `--embed-only` runs phase 2 alone and is always safe; it embeds only chunks that have no vector yet. `--limit N` caps phase 2 at N chunks, which is how you smoke-test an embedding change without committing hours to it.

```
shiaqa prune
```

Maintenance pass over an existing index: drops navigation and disambiguation pages and any vectors orphaned by earlier edits, *without* discarding embeddings. Prints the number of pages, chunks and orphan vectors removed.

```
shiaqa status
```

A table of raw-cache counts per namespace, indexed pages, chunks, vectors with percentage complete, build timestamp, Ollama health, database path and size, graph node and edge counts, graph build time, and the graph settings currently in force.

It also reports the graph as **stale** when `kg.db` is older than the newest file in the raw cache, so a graph built before a refetch cannot quietly look current.

5.2 Query commands

```
shiaqa search QUESTION [-k, --top-k INTEGER = 6] [--full]
```

Retrieve passages without generating an answer. No model call for generation is made, but the query is embedded, so Ollama must be reachable unless the index has no vectors.

Prints reasoning chains with their predicate trail, graph facts, then each hit as `[n] Title - Section (score, vec rank, bm25 rank)` with its URL and text. `--full` prints whole chunks instead of the first 220 characters. Exits 1 when nothing matches.

```
shiaqa ask QUESTION [-p, --profile TEXT] [-k, --top-k INTEGER]
                  [--json] [--decompose]
```

Ask a question and stream a cited answer.

`--profile` takes `fast`, `quality`, or any pulled Ollama tag passed straight through; the default comes from `gen_default_profile`. `--top-k` overrides `top_k_final` for this call. `--json` prints one JSON object (non-streaming) identical to the `/ask` response body. `--decompose` splits the question into at most `decompose_max_parts` sub-questions first, at the cost of one extra model call.

```
shiaqa serve [--host TEXT = "127.0.0.1"] [--port INTEGER = 8000] [--reload]
```

Run the HTTP service with uvicorn, serving `shiaqa.api:app`. `--reload` enables auto-reload for development. Binding to `0.0.0.0` exposes an unauthenticated service on the network — there is no auth layer in this project.

5.3 Knowledge-graph commands

Every subcommand below opens `data/kg.db` and exits 1 with “No graph yet. Run `shiaqa kg build`.” if it is missing. Titles are resolved by substring match, shortest label first; when several entities match, the others are listed so a wrong resolution is visible rather than silent.

```
shiaqa kg build [--jsonl-only]
```

Extract the graph from the raw cache into `data/kg/` and `data/kg.db`. Prints extraction counts, rows written per file, database counts and elapsed time.

`--jsonl-only` writes the JSONL files and the ontology snapshot but skips loading `kg.db`. Any override that has stopped matching is reported at the end under *Overrides needing attention*; that warning is never silenced, because a correction that has quietly stopped applying leaves the graph looking curated when it is not.

```
shiaqa kg stats [--top INTEGER = 12]
```

Node, edge, typed-edge and link counts, then three tables: entities by class, typed edges by predicate, and the unmapped infobox fields that carry links no predicate claims. That last table is the curation worklist — the top of it is where to extend `ontology.py`. `--top` sets rows per table.

```
shiaqa kg show TITLE [-n, --limit INTEGER = 25]
```

Everything the graph records about one article: its classes, its URL, up to `--limit` rendered one-line facts, and its categories. Facts are deduplicated, because the corpus states many of them from both ends.

```
shiaqa kg chain TITLE [-h, --hops INTEGER = 2] [-n, --limit INTEGER = 12]
                        [--deep-only]
```

Walk outward from one article and narrate what the graph connects it to. Each line carries its score, its hop count, the narrated sentence, the predicate trail (with `( rev )` marking an edge read backwards) and the infobox fields the steps came from.

`--deep-only` keeps only paths of two hops or more — the composed relations that exist in no single article. Uses `kg_beam_width` and `kg_hub_degree_max` from configuration.

```
shiaqa kg path START END [-h, --hops INTEGER] [-n, --limit INTEGER = 3]
```

How the graph connects two articles, strongest route first rather than merely shortest. `--hops` defaults to `kg_connect_max_hops`.

Exits 1 when there is no typed route, and says why that can be the right answer: routes through places and other hubs are refused on purpose, because a shared birthplace connects hundreds of unrelated people.

```
shiaqa kg compare FIRST SECOND [-n, --limit INTEGER = 8]
```

Line up what the graph records about two articles, predicate by predicate: each side's objects and, where they overlap, a *both* line. No traversal at all — one batched query, grouped by the rendered verb phrase. Exits 1 when the two share no populated predicate.

```
shiaqa kg viz [-o, --out TEXT] [--kind TEXT = "both"]
```

Write self-contained HTML views of the graph, defaulting to `<data_dir>/kg/viz`. `--kind` takes `explorer`, `schema` or `both`; anything else exits 1.

`explorer.html` shows one entity's typed neighbourhood at a time, with the infobox field each edge came from. `schema.html` shows the class-level shape, including the `(unclassified)` bucket, which is about a quarter of all edges and is deliberately not hidden. Each file embeds its own data, so both survive a strict content-security policy and open with no server.

6 HTTP API

FastAPI, four endpoints, one request model. Served by `shiaqa serve` ; OpenAPI at `/openapi.json` and Swagger UI at `/docs` .

The application object is `shiaqa.api:app` , titled *ShiaQAService*, version 0.1.0. A lifespan handler builds one `AnswerEngine` at startup, logs the index counts, and closes the engine's HTTP clients at shutdown. If no index exists at `db_path` the server still starts and logs a warning telling you to run `shiaqa fetch && shiaqa index` .

6.1 The request model

`AskRequest` is the body of all three POST endpoints.

FIELD	TYPE	DEFAULT	MEANING
<code>question</code>	<code>str</code> , 2–2000 chars	<i>required</i>	The question, in any language the generator handles.
<code>profile</code>	<code>str</code> <code>null</code>	<code>null</code>	<code>fast</code> , <code>quality</code> , or an explicit Ollama tag. <code>null</code> uses <code>gen_default_profile</code> .
<code>top_k</code>	<code>int</code> <code>null</code> , 1–20	<code>null</code>	Passages to keep after fusion. <code>null</code> uses <code>top_k_final</code> .
<code>decompose</code>	<code>bool</code> <code>null</code>	<code>null</code>	Split the question first, at the cost of one extra model call. <code>null</code> uses the <code>decompose</code> setting.

A value outside those bounds is rejected by Pydantic with `422 Unprocessable Entity` before any work is done.

6.2 Endpoints

GET /health → 200 application/json

Liveness plus a real readiness signal. `status` is `"ok"` only when Ollama is reachable with the embedding model pulled *and* every chunk has a vector; otherwise `"degraded"`, which is the honest state while embeddings are still building — BM25 and title matching already work, the semantic channel does not yet.

```
{
  "status":          "ok" | "degraded",
  "ollama":          str,           // "ok", or why not
  "index":            {"pages": int, "chunks": int, "vec_chunks": int},
  "vectors_complete": bool,
  "vectors_progress": float,        // percent, one decimal
  "models":           {"embed": str, "fast": str, "quality": str}
}
```

POST /search ← AskRequest → 200 application/json

Retrieval only, no generation. Fast, and the endpoint to tune against. `profile` is accepted and ignored.

```
{
  "question":         str,
  "retrieval_seconds": float,
  "context_tokens":   int,
  "chains":           [str],        // narrated multi-hop relations
  "kg_facts":         [str],        // one-line structured facts
  "sub_questions":    [str],        // non-empty only when decompose ran
  "hits": [
    {
      "n": int, "title": str, "section": str, "url": str,
      "kind": "summary" | "infobox" | "body",
      "score": float,           // fused RRF score, 5 dp
      "vec_rank": int | null,    // rank in the dense channel
      "bm25_rank": int | null,
      "text": str
    }
  ]
}
```

```
POST /ask <- AskRequest → 200 application/json
```

Retrieve, then generate one complete answer. Blocking: on CPU this is tens of seconds, so the streaming endpoint is usually the better choice for anything a person is waiting on. The body is `Answer.to_dict()`.

```
{
  "question": str,
  "answer": str,           // prose with [n] citations
  "model": str,           // the resolved Ollama tag
  "sources": [{"n": int, "title": str, "section": str,
               "url": str, "kind": str}],
  "stats": {
    "context_tokens": int,
    "retrieval_seconds": float, // 3 dp
    "generation_seconds": float, // 2 dp
    "output_tokens": int,
    "tokens_per_second": float // 1 dp
  }
}
```

When retrieval finds nothing, the endpoint still returns 200 with a plainly worded `answer` saying so and an empty `sources` list. No model call is made in that case.

```
POST /ask/stream <- AskRequest → 200 text/event-stream
```

The same work, streamed as Server-Sent Events: one `data:` line per event carrying compact JSON, terminated by a blank line. Response headers set `Cache-Control: no-cache` and `X-Accel-Buffering: no` so an intermediary cannot buffer the stream into uselessness.

Sources are sent *before* generation starts. That ordering is deliberate on this hardware: the caller can render where the answer will come from within a second while the model grinds on at 5–12 tokens per second.

6.3 Streaming event shapes

Exactly one `sources` event arrives first, then zero or more `delta` events, then exactly one `done` event.

TYPE	PAYLOAD
<code>sources</code>	<code>sources</code> (list, as in <code>/ask</code>), <code>chains</code> , <code>sub_questions</code> , <code>retrieval_seconds</code> , <code>context_tokens</code>
<code>delta</code>	<code>text</code> — the next piece of the answer, to be concatenated verbatim
<code>done</code>	<code>model</code> , <code>output_tokens</code> , <code>generation_seconds</code> (absent when nothing was generated)

```
curl -N -X POST localhost:8000/ask/stream \
  -H 'content-type: application/json' \
  -d '{"question": "Who compiled al-Kafi?", "profile": "fast"}'

data: {"type": "sources", "sources": [...], "chains": [], ...}

data: {"type": "delta", "text": "According"}

data: {"type": "done", "model": "llama3.2:3b", "output_tokens": 118, ...}
```

If retrieval finds nothing, the stream still emits its `sources` event (with an empty list), then a single `delta` carrying the “nothing found” text, then `done` with `output_tokens: 0`.

6.4 Status codes and errors

CODE	WHEN
200	Normal, including “nothing retrieved” and “degraded”.
422	Body fails validation — question too short or too long, <code>top_k</code> out of 1–20.
503	A request arrived before the lifespan handler finished building the engine.
500	Ollama refused or is unreachable. The upstream error is raised by <code>raise_for_status()</code> and surfaces as a server error; <code>/health</code> is the endpoint that reports this deliberately.

NO AUTHENTICATION

The service has no auth, no rate limiting and no CORS configuration. It is built to listen on `127.0.0.1`. Put it behind something before exposing it.

7 Python API

The importable surface, module by module. Every constructor takes an optional `Settings`, so nothing is bound to global state and tests inject their own.

```
# the two-line version: retrieve and answer
import asyncio
from shiaqa.rag.answer import AnswerEngine

async def main():
    engine = AnswerEngine()
    try:
        ans = await engine.answer("Who compiled al-Kafi?", profile="fast")
        print(ans.text)
        for src in ans.sources():
            print(src["n"], src["title"], src["url"])
    finally:
        await engine.aclose()

asyncio.run(main())
```

```
# retrieval alone, no event loop, no generation
from shiaqa.rag.retrieve import Retriever

res = Retriever().search_sync("Who compiled al-Kafi?", top_k=6)
print(res.context_tokens, [h.label for h in res.hits])
```

7.1 shiaqa.config and shiaqa.log

```
class Settings(BaseSettings)
def get_settings() -> Settings # lru_cache'd
```

Every tunable in the project. Reads `SHIAQA_`-prefixed environment variables and `.env`; unknown keys are ignored. `get_settings()` returns one cached instance and creates `data/`, `data/raw/` and `data/kg/` as a side effect. The full field list is §8.

```
def Settings.gen_model(profile: str | None = None) -> str
```

Resolve a profile to an Ollama tag. `fast / small / llama` → `gen_model_fast`; `quality / big / qwen` → `gen_model_quality`; anything else is passed through unchanged, which is what lets a caller name any pulled tag. `None` uses `gen_default_profile`.

Four paths are computed properties rather than fields, so they always follow `data_dir`: `raw_dir`, `db_path`, `kg_dir` and `kg_db_path`.

```
def get_logger(name: str = "shiaqa") -> logging.Logger
```

`shiaqa.log` — the one logger factory every module uses.

7.2 shiaqa.rag — answering

```
class AnswerEngine(settings: Settings | None = None,  
                   retriever: Retriever | None = None)
```

Retrieval plus generation against a local Ollama. Owns a lazily created `httpx.AsyncClient` with a 900-second timeout, because a 7B model on CPU can take minutes for one answer.

```
async def retrieve(question: str, top_k: int | None = None,  
                  decompose: bool | None = None) -> tuple[Retrieval, float]
```

Retrieve only, returning the result and the elapsed seconds. `decompose=None` falls back to the `decompose` setting; when it is on, the question is split and each part retrieved separately.

```
async def answer(question: str, profile: str | None = None,  
                 top_k: int | None = None,  
                 decompose: bool | None = None) -> Answer
```

One complete answer. When retrieval returns no hits, no model call is made and the returned `Answer` carries the fixed `NO_CONTEXT` text with an empty `hits` list.

```
async def stream(question: str, profile: str | None = None,  
                 top_k: int | None = None,  
                 decompose: bool | None = None) -> AsyncIterator[dict]
```

Yields the same events the SSE endpoint serialises: one `sources` dict, then `delta` dicts, then one `done`. The dicts are yielded as Python objects; the API layer is what turns them into `data:` lines.

```
async def aclose() -> None
```

Close the generation client and the embedder's clients. Always call it — the CLI does so in a `finally` block, and the API does so in its lifespan handler.

```
@dataclass class Answer
    question: str; text: str; model: str
    hits: list[Hit]; context_tokens: int
    retrieval_seconds: float; generation_seconds: float; eval_count: int

    @property tokens_per_second -> float
    def sources() -> list[dict]
    def to_dict() -> dict
```

`sources()` numbers the hits from 1, matching the `[n]` citation markers in `text`. `to_dict()` is exactly the `/ask` response body, with the timings already rounded.

7.3 shiaqa.rag.retrieve

```
class Retriever(settings: Settings | None = None,
                store: Store | None = None,
                embedder: Embedder | None = None,
                kg: KgStore | None = None)
```

Hybrid retrieval over four channels. The knowledge graph is attached automatically when `kg.db` exists and is populated *and* something asks for it (`kg_enabled` , a non-zero `weight_graph` , or `kg_facts_in_context`); otherwise `kg_active` is `False` and retrieval behaves exactly as it did before the graph existed.

```
async def search(question: str, top_k: int | None = None) -> Retrieval
def search_sync(question: str, top_k: int | None = None) -> Retrieval
```

The same pipeline twice, differing only in how the query embedding is fetched. Use `search_sync` from scripts and the CLI; use `search` inside an event loop.

Both run BM25 over FTS5, the title/alias channel, the dense channel (skipped entirely when the index holds no vectors) and the graph channel, fuse them with weighted RRF, diversify so that no article contributes more than `max_chunks_per_page` chunks, and pack the context to `context_token_budget` .

```
@property Retriever.kg_active -> bool
```

Whether a populated graph is attached to this retriever.

```
@dataclass class Retrieval
    question: str; hits: list[Hit]; context: str
    context_tokens: int; vector_used: bool
    kg_facts: list[str]; kg_chains: list[str]; kg_paths: list
    sub_questions: list[str]

    @property sources -> list[dict]
```

`context` is the assembled prompt context: an optional `[KG]` facts block, then an optional `[KG-PATH]` chain block, then the numbered excerpts. `kg_chains` and `kg_paths` are the same length and the same order — the strings reach the prompt, the objects keep the predicate and infobox field per step so a caller can show *why* a chain was drawn. A `kg_paths` entry is `None` where the line came from a comparison rather than a single walk. `sources` groups hits by URL, collecting their sections.

```
def to_fts_query(question: str) -> str
```

Turn a natural question into valid, forgiving FTS5 syntax: stopwords dropped, terms quoted, everything OR-ed. AND-ing a whole sentence returns nothing, and BM25 already rewards documents matching more of the rare terms.

Internally, `_rrf(rankings)` implements weighted reciprocal rank fusion with a per-channel RRF constant, and `_fuse(...)`, `_graph_channel(...)`, `_graph_paths(...)`, `_graph_chains(...)`, `_graph_facts(...)` and `_pack_context(...)` are the stages a retrieval change usually touches.

7.4 shiaqa.rag.decompose

```
Chat = Callable[[str], Awaitable[str]]

class Decomposer(chat: Chat, max_parts: int = 3)
    async def split(question: str) -> list[str]
```

Splits a question into independent factual sub-questions, given any async `chat(prompt) -> str` callable. The callable is injected rather than reached for through `AnswerEngine`, so tests exercise every failure mode without an HTTP client or an Ollama.

Every failure path returns `[]`: a timeout, prose instead of a list, an answer instead of a split, or a single sub-question. The caller then runs the ordinary single-shot flow, so decomposition can never break the default path.

```
def parse(raw: str, max_parts: int = 3) -> list[str]
```

Pull sub-questions out of whatever the model actually said: numbered lines, bullets, code fences and preambles are all handled. Returns `[]` for anything unusable, including a single sub-question — which is not a decomposition and would cost a model call to learn nothing.

7.5 shiaqa.store.sqlite_store

One SQLite file holds all three indexes — `chunks` rows, an FTS5 BM25 index and a `sqlite-vec` vector table — written in the same transaction so the lexical and semantic views cannot drift apart.

```
class Store(settings: Settings | None = None, path: Path | None = None)
```

Opens the database, loads the `sqlite-vec` extension and sets WAL journaling. `path` overrides `settings.db_path`, which is how tests point at a temporary file.

```
@dataclass class Hit
    chunk_id: int; page_id: int; title: str; url: str
    section: str; kind: str; text: str; tokens: int; score: float
    vec_rank: int | None = None; bm25_rank: int | None = None

    @property label -> str # "Title - Section", or just "Title"
```

One retrieved chunk with its provenance and its rank in each channel that found it. `kind` is `summary`, `infobox` or `body`.

LIFECYCLE

```
def init_schema() -> None
def reset() -> None # DESTRUCTIVE – drops every table, vectors included
def close() -> None
def commit() -> None
def set_meta(key: str, value: str) -> None
def get_meta(key: str, default: str | None = None) -> str | None
def counts() -> dict[str, int] # pages, chunks, vec_chunks
```

`reset()` drops `vec_chunks`, `chunks_fts`, `chunks`, `pages` and `meta`, then recreates the schema. It is the call behind the destructive `shiaqa index`. Known meta keys are `embed_model`, `built_at` and `namespaces`.

WRITING

```
def add_page(page, aliases: list[str]) -> None
def add_chunks(chunks: Iterable[Chunk], aliases: str = "") -> list[int]
def add_vectors(pairs: Sequence[tuple[int, Sequence[float]]]) -> None
def delete_page(page_id: int) -> int
def chunks_missing_vectors() -> list[sqlite3.Row]
```

`add_chunks` writes the row and its FTS5 entry together and returns the new chunk ids. Aliases ride along in the lexical index only, so a query for *Abu Turab* finds the Imam Ali article without polluting the text that gets embedded or shown to the model.

`delete_page` returns the number of chunks removed. Use it rather than a bare `DELETE`: `chunks_fts` is contentless, so FTS5 rows must be retired with the special `'delete'` command replaying the indexed values, and a plain delete raises `OperationalError`.

`chunks_missing_vectors()` is what makes phase 2 resumable, and `add_vectors` expects vectors already L2-normalised — that is what makes `sqlite-vec`'s Euclidean ranking equivalent to cosine.

READING

```
def vector_search(embedding: Sequence[float], k: int) -> list[tuple[int, float]]
def bm25_search(query: str, k: int) -> list[tuple[int, float]]
```

`(chunk_id, distance)` nearest neighbours, and `(chunk_id, score)` BM25 hits with column weights title 6.0, section 2.0, aliases 4.0, text 1.0. Malformed FTS5 syntax returns `[]` rather than raising, so a user's question can never crash retrieval.

```
def title_candidates(terms: set[str], fts_query: str,
                    pages: int, per_page: int) -> tuple[list[int], list[int]]
def title_search(terms: set[str], fts_query: str,
                pages: int, per_page: int) -> list[int]
```

Chunks from the articles a question actually *names*, plus the page ranking behind them. Recall comes from FTS over titles and 35k redirect aliases; precision comes from re-scoring each candidate on *coverage* — how much of the article's own name the question accounts for — with redirect count as a prominence prior to break ties between same-named figures. Matched articles contribute their fact box and summary first, round-robin so one page cannot own the list.

The page ids come back because the graph channel seeds from exactly these pages; re-running the 600-row recall query to rediscover them would be pure waste. `title_search` is the older single-value form.

```
def page_chunks(page_ids: Sequence[int], per_page: int,
                kinds: Sequence[str] = ("infobox", "summary")) -> list[int]
def hydrate(scored: Sequence[tuple[int, float]],
            ranks: dict[int, tuple[int | None, int | None]] | None = None) -> list[Hit]
```

`page_chunks` returns lead chunks for pages reached indirectly — fact box before summary, round-robin across pages. `hydrate` turns scored chunk ids into `Hit` objects, attaching the per-channel ranks when given.

```
def normalise_terms(text: str) -> set[str]
```

Fold a title or a query into comparable bare terms: lowercased, parenthetical qualifiers dropped, diacritics stripped, noise words removed. This is what lets *Husayn* and *Husayn* compare equal.

7.6 shiaqa.kg.store

`data/kg.db` — the derived, queryable side of the graph. Rows reference `page_id`, never `chunk_id`: MediaWiki page ids survive a reindex, and an autoincrement `chunks.id` does not.

```
class KgStore(settings: Settings | None = None, path: Path | None = None)
    def init_schema() / reset() / close() -> None
    def set_meta(key, value) / get_meta(key, default=None)
    def counts() -> dict[str, int] # nodes, edges, links, typed_edges
    @property is_populated -> bool
    def load(nodes: Iterable[dict], edges: Iterable[dict],
            links: Iterable[dict]) -> None
```

Tables are `nodes`, `node_classes`, `edges`, `links` and `meta`. Both edge directions are indexed, which is what lets traversal read an edge backwards instead of materialising an inverse row for every fact. `reset()` here is safe — everything is re-derived from the JSONL in seconds.

```
def node(node_id: str) -> sqlite3.Row | None
def node_for_page(page_id: int) -> str | None
def find_by_label(label: str, limit: int = 10) -> list[sqlite3.Row]
def classes_of(node_id: str) -> list[str]
def classes_for(node_ids: Sequence[str]) -> dict[str, list[str]]
```

Node ids are `page:<page_id>` for entities, plus class and date nodes. `find_by_label` is a substring match ordered by label length, so the shortest — usually the canonical — match comes first. `classes_for` is the batched sibling of `classes_of`; batching matters because traversal asks about dozens of nodes at once.

```
def edges_from(node_ids: Sequence[str],
               include_taxonomy: bool = False) -> dict[str, list[dict]]
def edges_of(node_id: str, include_taxonomy: bool = False) -> list[dict]
```

Every typed edge touching each node, one query pair for the whole batch. Each edge is `{"dir": "out"|"in", "p", "other", "label", "kind", "page_id", "field"}`, where `field` is the infobox field it came from and `page_id` is carried straight off the join — that field is the whole point, because it lets a hop be ranked without a `node()` lookup per edge.

Results are sorted by predicate priority, and the SQL `ORDER BY` is load-bearing rather than cosmetic: ties on priority decide which neighbours survive a beam, so stating the order makes the ranking a contract instead of an index artefact. `include_taxonomy=True` also returns `instance_of` and `subclass_of`.

```
def degrees(node_ids: Sequence[str]) -> dict[str, int]
def prominence(node_ids: Sequence[str]) -> dict[str, int]
```

`degrees` is typed degree in *both* directions — the fan-out traversal actually faces. `prominence` adds redirect count to that degree and is a tie-break, not a score: it is how the walker chooses between two equally scored paths. Note it rewards the same property `hub_degree_max` refuses to travel through, which is consistent rather than contradictory — prominence is good to land on and ruinous to pass through.

```
def neighbour_pages(seed_node_ids: Sequence[str], limit: int) -> list[int]
def facts(node_id: str, predicates: dict, limit: int) -> list[str]
```

`neighbour_pages` returns page ids one typed hop from the seeds, best predicates first, round-robin across seeds. This is the one-hop graph channel, kept exactly as it was so that `kg_max_hops = 1` is byte-identical to the pre-multi-hop system.

`facts` renders a node's edges as readable one-line statements, deduplicated — the same fact is often asserted from both ends, as when al-Kulayni's `works` names al-Kafi and al-Kafi's `author` names al-Kulayni. `predicates` is `Ontology().predicates`.

7.7 shiaqa.kg.paths

Multi-hop traversal. All of it is pure SQLite — no model calls — and costs about 10 ms at retrieval scale, which is what makes it viable on a CPU-only box.

```
@dataclass(frozen=True) class Step
    p: str; direction: "out" | "in"; src: str; dst: str
    dst_label: str | None; dst_kind: str | None      # entity | class | date
    dst_page_id: int | None; field: str | None      # the infobox field

    @property stored -> tuple[str, str, str]      # the edge as kg.db holds it
    def as_edge() -> dict                        # the edges_of shape, for render
```

One traversed edge, with everything needed to rank it or explain it. `direction` is relative to the node departed from, so `stored` puts the triple back the way the database holds it regardless of which way the walk went.

```
@dataclass(frozen=True) class Path
    seed: str; seed_label: str; steps: tuple[Step, ...]
    score: float; shape: str = "chain"            # chain | connect

    @property hops / end / end_label / end_page_id / nodes
```

A scored route. `shape` distinguishes a walk outward from a route found between two named articles; the distinction matters to callers, because for a `connect` path the interesting article is usually the *middle*, not the end — the end is the other article the question already named.

```
def expand(kg: KgStore, seed_node_ids: Sequence[str], predicates: dict, *,
           max_hops: int = 2, beam: int = 24, limit: int = 12,
           no_transit: frozenset[str] = NO_TRANSIT,
           hub_degree_max: int = HUB_DEGREE_MAX) -> list[Path]
```

Best-scoring paths up to `max_hops` out from the seeds. The final selection is round-robin across *both* seed and hop depth, not score-ordered: every one-hop path outranks every two-hop path by construction, so a flat truncation would spend the whole budget on first-hop articles and leave the chain block empty precisely when it matters.

```
def connect(kg: KgStore, a: str, b: str, predicates: dict, *,
            max_hops: int = 3, beam: int = 24, limit: int = 3,
            no_transit: frozenset[str] = NO_TRANSIT,
            hub_degree_max: int = HUB_DEGREE_MAX) -> list[Path]
```

Typed paths between two named entities, *ranked* rather than merely shortest. That matters more than it sounds: this corpus joins al-Mufid and al-Tusi at length two both through a shared teacher and through a shared city, and a shortest-path search returns whichever it reaches first. Longer routes between the same pair are dropped as detours rather than offered as alternatives.

```
def compare(kg: KgStore, a: str, b: str, predicates: dict, *,
            limit: int = 6) -> Comparison | None
```

Predicates asserted of both entities, aligned side by side. No traversal — one batched call, grouped by the rendered verb phrase so that reciprocal pairs fold together. Returns `None` when the two share no populated predicate, which is data rather than a judgement call. A `Comparison` holds `a`, `a_label`, `b`, `b_label`, `rows`; each `ComparisonRow` holds `p`, `direction`, `label`, `a_objects`, `b_objects`, `shared`.

```
def narrate(path: Path, predicates: dict,
            persons: set[str] | None = None) -> str | None
def narrate_comparison(cmp: Comparison, limit_objects: int = 3) -> list[str]
def person_nodes(kg: KgStore, node_ids: Sequence[str]) -> set[str]
```

`narrate` renders a path as one English sentence, chaining with a relative pronoun while the steps read forwards. A step that cannot be read from the anchor's side *breaks* the sentence into a new clause rather than being chained: 53 of the 69 predicates declare no inverse, and chaining one regardless would state the relation backwards and put a quietly false sentence in the prompt.

`person_nodes` tells the narrator which nodes are people, so it can choose “who” over “which”.

```
def tier(p: str) -> int
def score_steps(steps: Sequence[Step]) -> float
def canonical(stored: tuple[str, str, str], predicates: dict) -> tuple[str, str, str]
```

`score_steps` is hop decay times the weight of every predicate travelled. `canonical` folds an edge with its reciprocal onto one key: the corpus states 1,278 facts from both ends, and a beam that admits both spends two slots on one relation and halves its own width.

TWO GUARDS, AND NEITHER IS REDUNDANT

`NO_TRANSIT` names predicates a chain may land on but never continue through — without it, *al-Mufid born_in Baghdad studied_in al-Tusi* is a real two-hop path on this corpus, and it is worthless. `hub_degree_max` stops a chain routing through a node of high degree. Places need the first; the biggest hubs are *people* reached by `taught_by`, `companion_of` and `father` — exactly the predicates multi-hop questions turn on, which can never be refused by name. Removing either guard leaves the other looking sufficient right up until it is not.

7.8 shiaqa.kg — build, ontology, render, viz

```
def build(settings: Settings | None = None,
          jsonl_only: bool = False) -> BuildReport
def load_db(settings: Settings | None = None) -> dict[str, int]
def read_jsonl(path: Path) -> Iterator[dict]
```

Full rebuild, cheap enough (~15 s) that there is no resumable phase. Writes `nodes.jsonl`, `edges.jsonl`, `links.jsonl`, `unmapped.jsonl` and `ontology.json`, then rebuilds `kg.db` unless `jsonl_only`. `BuildReport` carries `extract`, `files`, `db_counts`, `seconds` and `overrides`.

```
class Extractor(settings=None, ontology=None, overrides=None)
    def run() -> Extraction
    def resolve(raw_target: str) -> str | None
    def node_for(title: str) -> str | None
    def resolve_node(title: str) -> str | None
```

The extraction pass over `data/raw/`. `Extraction` holds `nodes`, `edges`, `links`, `unmapped`, `report` and `overrides`.

This is deliberately a *second* pass rather than a hook into ingest: `clean._parse_infobox` flattens `[[Imam al-Mahdi (a)]]` to bare text and discards the target, and widening that return type would change the `kind='infobox'` chunk text that is already embedded — silently invalidating every stored vector.

```
class Ontology()
    .classes: dict[str, tuple[str, ...]]
    .predicates: dict[str, Predicate]
    def classes_for(template: str) -> list[str]
    def predicate_for(raw_field: str, subject_classes: set[str]) -> Predicate | None
    def is_noise(raw_field: str) -> bool
    def snapshot() -> dict
```

The curation: 17 classes and 69 predicates, mapped from infobox template and field names. It **validates itself on construction** — a predicate whose `inverse` does not exist, or a field claimed by two predicates without disjoint domains, raises `ValueError` at import rather than silently mis-typing edges.

A `Predicate` is `name`, `label`, `fields`, `range ("entity"|"date")`, `inverse`, `domain`, `range_classes`. Extend the table from the top of `data/kg/unmapped.jsonl`, which lists every infobox field carrying links that no predicate claims.

```
@dataclass(frozen=True) class EdgeOverride
    action: "drop" | "add"; subject: str; predicate: str
    object: str | None; note: str

def validate(overrides, predicates: Iterable[str]) -> None
def apply(edges: list[dict], resolve: Callable[[str], str | None],
          overrides = OVERRIDES,
          predicates: Iterable[str] | None = None) -> tuple[list[dict], OverrideReport]
```

Single-edge corrections, written by article title rather than page id so a person can read them. For facts that are wrong at the source and cannot be expressed as a field mapping: WikiShia lists Nahj al-balagha under a *calligrapher's* works because he transcribed it, which no range check can catch — the types are right and only the meaning is wrong.

An override with no `note`, an unknown predicate, or an `add` without an object is rejected at build time. One that stops matching is reported as **stale** by `kg build` and `kg stats`.

NEVER HAND-EDIT DATA/KG/*.JSONL OR DATA/KG.DB

Both are generated and gitignored: an edit there is destroyed by the next build and was never reviewable. That is why `overrides.py` lives in `src/`. Use `ontology.py` when a whole field is mis-mapped, and `overrides.py` when exactly one edge is wrong.

```
def clause(edge: dict, predicates: dict) -> tuple[str, str] | None
def flipped(edge: dict, predicates: dict) -> tuple[str, str] | None
def sentence(subject: str, edge: dict, predicates: dict) -> str | None
```

`shiaqa.kg.render` — turning one edge back into one English clause, in one place with two callers (`KgStore.facts` and path narration). `clause` returns `None` when the sentence cannot start at the anchor: an unmapped predicate, an unlabelled far end, or an inbound edge whose predicate declares no inverse. `flipped` is the fallback that starts the sentence at the other end instead.

```
def write(out_dir: Path, kinds: tuple[str, ...] = ("explorer", "schema"),
          settings: Settings | None = None) -> dict[str, Path]
def explorer_payload(kg, onto=None, settings=None) -> dict
def schema_payload(kg, onto=None) -> dict
def render(template: str, payload: dict, title: str) -> str
```

`shiaqa.kg.viz` — self-contained HTML views. Each page embeds its own data (the typed graph trims to about 540 KB), so it opens from disk with no server. Both views collapse reciprocal edges the way `KgStore.facts()` does.

```
def parse_one(text: str) -> DateValue
def parse_field(raw_value: str, resolved_titles: list[str]) -> DateValue

@dataclass(frozen=True) class DateValue
    ah_year / ah_month / ah_day / ce_year / ce_month / ce_day / bh_year
    @property node_id -> str; @property label -> str; def as_node() -> dict
```

`shiaqa.kg.dates` — dates in whichever calendars the source supplied, Hijri, Gregorian or Before Hijra. An empty `DateValue` is falsy and has no node id; check `bool(dv)` before asking for one.

7.9 shiaqa.ingest and shiaqa.wiki

```
def build_text_index(settings=None, store=None) -> IndexReport # DESTRUCTIVE
def build_vectors(settings=None, store=None, limit: int | None = None) -> IndexReport
def prune(settings=None, store=None) -> dict[str, int]
def is_navigation_page(raw: dict) -> bool
```

`shiaqa.ingest.pipeline`. Phase 1 calls `Store.reset()` first — that is the destructive command of §3.4. Phase 2 processes only chunks without a vector, so it is resumable, and it raises `RuntimeError` up front if Ollama is not healthy rather than failing partway through. `IndexReport` carries `pages`, `skipped`, `chunks`, `embedded`, `seconds`.

Disambiguation and navigation pages are skipped at index time: they are lists of links with no prose, and they outrank real articles on title match.

THE EMBEDDING TEXT FORMAT IS DUPLICATED

`build_vectors()` reconstructs the context header inline rather than calling `Chunk.embed_text()`, because it reads database rows, not `Chunk` objects. If you change one, change the other — otherwise stored vectors no longer match the representation queries are embedded against, and retrieval degrades with no error.

```
def clean_page(raw: dict) -> CleanPage
def infobox_as_text(title: str, infobox: dict[str, str]) -> str

@dataclass class CleanPage
    pageid, ns, title, url, revid, timestamp, summary
    sections: list[Section]; infobox: dict[str, str]
    categories: list[str]; pointers: list[str]; citations: list[str]
    @property char_len -> int
```

`shiaqa.ingest.clean` — wikitext to structured prose. A `Section` is `path`, `level`, `text` with a `label` property that joins the path with `>`, or "Summary" for the lead.

ALWAYS PARSE WIKITEXT THROUGH CLEAN._PARSE()

Never a bare `mwparserfromhell.parse()` . `_parse` sets `skip_style_tags=True` ; transliterated Arabic leaves the italic state machine unbalanced, after which the parser silently stops recognising *every* template in the block and infoboxes leak into prose as raw markup.

```
def chunk_page(page: CleanPage, settings: Settings | None = None) -> list[Chunk]
def estimate_tokens(text: str) -> int
def chunk_stats(chunks: list[Chunk]) -> dict[str, float]

@dataclass class Chunk
    page_id, ns, title, url, section, ordinal
    kind: "summary" | "infobox" | "body"; text: str; tokens: int
    @property context_header -> str
    def embed_text() -> str; def cite_label() -> str
```

`shiaqa.ingest.chunk` — section-aware chunking. Each chunk is embedded with a context header (Article – Section > Subsection), because passages on this wiki constantly say “he was martyred at Karbala” without naming whose biography it is.

`estimate_tokens` divides by `CHARS_PER_TOKEN = 2.6` , measured against llama3.2 and qwen2.5 on this corpus rather than the usual ~4: transliteration with diacritics tokenises far denser. Do not “correct” it.

```
async def fetch_all(settings=None, namespaces: list[int] | None = None,
                    force: bool = False) -> dict[int, int]
async def fetch_namespace(client, namespace, settings, force=False) -> int
def read_raw(settings=None, namespaces=None) -> Iterator[dict]
def raw_counts(settings=None) -> dict[int, int]
def raw_path(settings, namespace: int) -> Path
def load_redirects(settings=None) -> dict[str, str]
def load_aliases(settings=None) -> dict[str, list[str]]
```

`shiaqa.wiki.fetch` . Each namespace is written to a `.jsonl.part` file and renamed on completion, so an interrupted fetch never leaves a half-file that looks complete. `load_redirects` maps redirect title to target; `load_aliases` inverts that into target to alias list, which is what feeds the title channel.

```

class WikiClient(settings: Settings | None = None)      # async context manager
    async def get(**params) -> dict
    async def paged(**params) -> AsyncIterator[dict]
    async def list_pages(namespace: int) -> list[dict]
    async def fetch_batch(titles: list[str]) -> list[dict]
    async def redirect_map(namespace: int = 0) -> dict[str, str]

class WikiError(RuntimeError)

```

`shiaqa.wiki.client` . Identifies itself with a contact address, serialises requests, sleeps between calls and honours `maxlag` . `fetch_batch` respects the 50-title anonymous API cap. Do not parallelise it.

7.10 shiaqa.embed and shiaqa.rag.prompt

```

class Embedder(settings: Settings | None = None)
    def embed_documents(texts: Sequence[str]) -> list[list[float]]
    def embed_query_sync(text: str) -> list[float]
    async def embed_query(text: str) -> list[float]
    def health() -> tuple[bool, str]
    def close() -> None; async def aclose() -> None

```

Embeddings via a local Ollama, POSTed to `/api/embed` . Vectors are L2-normalised on the way in, which is what makes `sqlite-vec` 's Euclidean ranking identical to the cosine ranking `omic-embed-text` is trained for.

The model is asymmetric: documents get `embed_doc_prefix` , queries get `embed_query_prefix` . Mixing them silently costs accuracy, which is why the two call sites are separate methods rather than one with a flag. `health()` returns `(False, reason)` when Ollama is unreachable or the embedding model is not pulled.

```

def build_messages(question: str, context: str) -> list[dict[str, str]]

```

```

SYSTEM: str; USER_TEMPLATE: str; NO_CONTEXT: str

```

`shiaqa.rag.prompt` . Two constraints shape this prompt. The generator is a 3B or 7B model on CPU, so the instructions are short, concrete and positively phrased. And WikiShia is a confessional (Twelver Shia) encyclopaedia, so the system prompt deliberately instructs attribution — “According to WikiShia...” — rather than neutral assertion. **Preserve that framing in any user-facing output you build on top of this.**

The rules also require citing an excerpt number after each claim, saying plainly when the excerpts do not contain the answer, keeping dates in the form the excerpts use, and answering in the question's language.

8 Configuration reference

Every field of `Settings`, its default, and what changing it costs. Override any of them with a `SHIAQA_`-prefixed environment variable or a line in `.env`.

```
SHIAQA_TOP_K_FINAL=8 .venv/bin/shiaqa search "... "      # one call
echo "SHIAQA_GEN_DEFAULT_PROFILE=quality" >> .env        # every call
```

Prefer adding a setting here over hard-coding a constant. Note that `get_settings()` is `lru_cache`d, so environment changes made after the first call within one process have no effect.

8.1 Data source and storage

FIELD	DEFAULT	NOTES
<code>wiki_api</code>	<code>https://en.wikishia.net/w/api.php</code>	MediaWiki endpoint.
<code>wiki_view_base</code>	<code>https://en.wikishia.net/view/</code>	Base for the citation URLs.
<code>namespaces</code>	<code>[0, 3000, 14]</code>	Articles, <code>Text:</code> sources, categories.
<code>user_agent</code>	identifies the client + contact	Operators ask for a contact address. Keep it honest.
<code>request_delay</code>	<code>0.35</code>	Seconds between API calls.
<code>maxlag</code>	<code>5</code>	Back off when the wiki's replicas fall behind.
<code>batch_titles</code>	<code>50</code>	API cap for anonymous users.
<code>data_dir</code>	<code><project>/data</code>	<code>raw_dir</code> , <code>db_path</code> , <code>kg_dir</code> , <code>kg_db_path</code> all follow it.

8.2 Models

FIELD	DEFAULT	NOTES
ollama_host	http://127.0.0.1:11434	
embed_model	nomic-embed-text	Changing it invalidates every stored vector.
embed_dim	768	Must match the model; fixes the vec0 table width.
embed_doc_prefix	"search_document: "	The model is asymmetric. Mixing the two silently costs accuracy.
embed_query_prefix	"search_query: "	
embed_batch	16	Texts per /api/embed call.
gen_model_fast	llama3.2:3b	~14 s per answer.
gen_model_quality	qwen2.5:7b-instruct-q4_K_M	~27 s per answer.
gen_default_profile	fast	Used when a request names no profile.
gen_num_ctx	8192	Ollama context window.
gen_temperature	0.2	
gen_max_tokens	700	Hard cap on answer length.

8.3 Chunking

FIELD	DEFAULT	NOTES
chunk_target_tokens	380	Token counts are estimated at 2.6 characters per token, measured on this corpus. Changing any of these requires a full reindex, embeddings included.
chunk_max_tokens	620	
chunk_overlap_tokens	60	
chunk_min_tokens	25	

8.4 Retrieval

FIELD	DEFAULT	NOTES
top_k_vector	30	Candidates from the dense channel.
top_k_bm25	30	Candidates from BM25.
top_k_final	6	Passages kept after fusion.
rrf_k	60	RRF constant for the two long, noisy channels.
weight_vector	1.0	
weight_bm25	1.0	
weight_title	1.4	Highest: on a wiki, a question that names an article is usually answered by it.
rrf_k_title	20	Smaller because the channel is short and precision-ordered. At 60, a fact box at title-rank 1 lost to mid-list chunks.
title_pages	4	Named articles considered.
title_chunks_per_page	3	
context_token_budget	2600	On CPU, prompt length dominates latency.
max_chunks_per_page	3	Stops one long article owning the context window.

8.5 Knowledge graph

FIELD	DEFAULT	NOTES
kg_enabled	False	Forces the graph open and warns when <code>kg.db</code> is missing. The graph is used anyway whenever <code>weight_graph > 0</code> .
weight_graph	0.8	Mid-plateau: 0.5, 0.8 and 1.0 score identically. <code>0.0</code> makes retrieval byte-identical to the pre-graph system; <code>2.0</code> drowns the other three channels.
rrf_k_graph	20	Same reasoning as <code>rrf_k_title</code> .
kg_neighbour_pages	6	Articles the channel may contribute.
kg_chunks_per_page	2	
kg_facts_in_context	False	A <code>[KG]</code> facts block mostly restates articles the title channel already retrieved.
kg_facts_max	12	
kg_max_hops	2	<code>1</code> is byte-identical to the pre-multi-hop system, by keeping that branch on the old call.
kg_beam_width	24	Frontier cap per level — a precision device, not a performance one.
kg_path_limit	12	Paths kept per expansion.
kg_hub_degree_max	40	A node above this may end a path but never continue one.
kg_connect_max_hops	3	Longest route between two named articles.
kg_chains_in_context	True	On by default: a chain carries a relation that exists in <i>no</i> chunk. Inert when <code>kg_max_hops</code> is 1.
kg_chains_max	4	Lines in the <code>[KG-PATH]</code> block.

8.6 Question decomposition

FIELD	DEFAULT	NOTES
decompose	False	Off because it is the only part of multi-hop that costs model calls.
decompose_max_parts	3	Sub-questions kept.
decompose_num_predict	160	Output cap for the splitting call.

9 Evaluating retrieval changes

Run this after any retrieval change. It is the only thing standing between a plausible idea and a quiet regression.

```
.venv/bin/python eval/run_eval.py --k 6
.venv/bin/python eval/run_eval.py --k 6 --sweep           # grid-search fusion settings
.venv/bin/python eval/run_eval.py --k 6 --kg             # graph channel off vs on
.venv/bin/python eval/run_eval.py --k 6 --hops           # one hop vs two, all three sets
.venv/bin/python eval/run_eval.py --k 6 --kg --weight-graph 1.0
```

9.1 The three question sets

FILE	N	WHAT IT MEASURES
eval/questions.yaml	15	Regression. Each names the article(s) that must be retrieved; five also assert a string that must appear in the assembled context.
eval/questions_kg.yaml	10	Questions that turn on a <i>relation</i> rather than a passage — the case the flat channels are worst at.
eval/questions_multihop.yaml	12	Questions needing <i>two</i> typed hops. <code>expect</code> names only the far article, never the seed.

That last design rule is what makes the multi-hop set measure anything. Naming the seed as well would read `recall@6 1.00` at every setting, because scoring takes any match in `expect` and the title channel finds the seed at rank 1.

9.2 Metrics, and the one that lies

The suite reports `recall@k`, MRR, answer-in-context and median retrieval latency. The metric that matters most is **answer-in-context**: retrieving the right *article* is worthless if the chunk holding the fact never reaches the prompt.

`--hops` reports answer-in-context **twice** — over the whole prompt, and over the retrieved passages with the `[KG]` and `[KG-PATH]` blocks stripped. The second exists because the first stops measuring anything once chains are on: a chain stating “al-Radi was taught by al-Mufid, who was taught by al-Saduq” trivially satisfies a search for “al-Saduq”. Reading the gap between the two separates *we retrieved the right passage* from *we asserted the fact*.

9.3 Current baseline

SET	RECALL@6 / MRR / ANSWER-IN-CONTEXT
Regression (15)	1.00 / 0.933 / 1.00 , ~109 ms median retrieval
Relational (10)	1.00 / 0.950 / 0.90
Multi-hop (12)	answer-in-context 0.75 ; in-excerpts 0.50

Read the multi-hop numbers honestly. Most of the gain from two hops is the chain block putting the composed relation in the prompt. The retrieval gain is real but smaller, and recall@6 barely moves — the far article still rarely wins a top-6 slot, because the seed takes three of the six under `max_chunks_per_page` and nothing here ranks a relation by whether the question actually asked about it. Do not tune `weight_graph` against the answer-in-context figure; it is not measuring the channel.

TREAT THE SWEEP WITH SUSPICION

Fifteen questions are enough to catch a structural bug and not enough to justify finely-tuned weights. The same applies to `HOP_DECAY` , `TIER_WEIGHT` and `kg_hub_degree_max` , which are set from graph structure rather than fitted to twelve questions. The current settings deviate from the defaults in exactly one parameter, chosen because its effect held across the whole grid.

10 Troubleshooting

SYMPTOM	CAUSE AND FIX
<code>/health</code> says <code>degraded</code>	Either Ollama is unreachable / the embedding model is not pulled (see the <code>ollama</code> field), or embeddings are incomplete (see <code>vectors_progress</code>). Finish them with <code>shiaqa index --embed-only</code> .
Requests return <code>503</code>	The lifespan handler has not finished building the engine. Retry in a moment.
cannot reach Ollama at...	Ollama is not running, or is on another host. Start it, or set <code>SHIAQA_OLLAMA_HOST</code> .
<code>model ... not pulled</code>	<code>ollama pull nomic-embed-text</code> .
Answers say nothing was found	No index, or the question uses spellings the corpus does not carry. Check <code>shiaqa status</code> , then try <code>shiaqa search</code> to see what retrieval is actually returning.
Retrieval got worse after a change	Run <code>eval/run_eval.py --k 6</code> against the regression set before doing anything else. If the change touched the embedding text format, both copies must match (§7.9).
<code>kg</code> is enabled but <code>data/kg.db</code> is missing	<code>shiaqa kg build</code> . It takes about fifteen seconds and cannot disturb the index.
<code>shiaqa status</code> says the graph is stale	The raw cache is newer than <code>kg.db</code> . Rebuild the graph.
<code>kg path</code> finds no route	Often correct rather than a failure: routes through places and high-degree hubs are refused on purpose. Try <code>kg chain</code> on each end, or raise <code>--hops</code> .
<code>OperationalError</code> deleting chunks	<code>chunks_fts</code> is contentless. Use <code>Store.delete_page()</code> , never a bare <code>DELETE</code> .
Answers are slow	Expected: 14 s on <code>fast</code> , 27 s on <code>quality</code> , plus ~7 s for <code>--decompose</code> . Use <code>/ask/stream</code> so sources appear immediately, and keep <code>context_token_budget</code> tight.
Infoboxes appear as raw markup in a chunk	Wikitext was parsed without <code>skip_style_tags</code> . Always go through <code>clean._parse()</code> .

11 Licensing and attribution

The Apache-2.0 licence in this repository covers **code only**, not the WikiShia content the tool downloads. WikiShia publishes no machine-readable licence — its API `rightsinfo` is empty — so do not assume CC BY-SA for the corpus. See `NOTICE` in the repository root before redistributing anything derived from the fetched text.

`data/` is gitignored and holds roughly 250 MB: the raw wikitext cache, the built index and the knowledge graph. Never commit it. Everything in it except the raw cache can be re-derived offline; only `shiaqa fetch` needs the network.

ATtribution is a design requirement, not a default

WikiShia is a confessional (Twelver Shia) encyclopaedia. The system prompt deliberately instructs attribution — “According to WikiShia...” — rather than neutral assertion, and asks the model to note where the excerpts say Sunni and Shia accounts differ. Preserve that framing in any interface built on this API.

Be a polite API client. `wiki/client.py` identifies itself with a contact address, serialises its requests, honours `maxlag` and caches everything. Do not parallelise the fetcher.

11.1 Where to look next

FILE	WHAT IT HOLDS
<code>README.md</code>	Project overview, corpus statistics and the reasoning behind the retrieval design.
<code>CLAUDE.md</code>	Working notes for contributors: the destructive operations, the invariants that fail silently, and how each tuned number was chosen.
<code>src/shiaqa/config.py</code>	Every tunable, with the evidence for its default in the comments.
<code>src/shiaqa/kg/ontology.py</code>	The 17 classes and 69 predicates, and the infobox fields each one claims.
<code>data/kg/unmapped.jsonl</code>	The curation worklist: infobox fields carrying links no predicate claims yet.
<code>tests/</code>	171 offline tests. The fastest way to learn what a function guarantees.

ShiaQAService user manual, version 0.1.0. Generated from `doc/user-manual.html` ; signatures verified against the source at the time of writing. If a signature here disagrees with the code, the code is right — and this file needs a line.